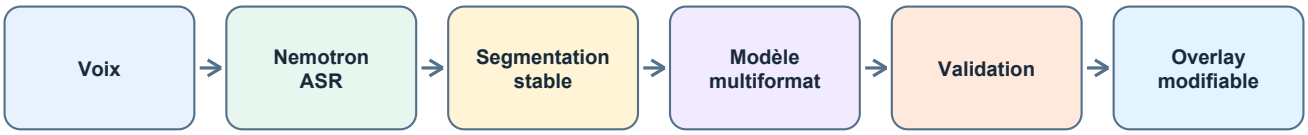
DictAI : fine-tuning multiformat et correction pendant la dictée

Recherche et spécification du 19 septembre 2026. Ce document décrit le corpus V4, le pilote Gemma 4 E2B et le fonctionnement Android à réaliser. Les résultats d'exécution du pilote sont consignés séparément dans le projet de fine-tuning. Le traitement progressif décrit ici n'est pas encore activé dans Android.

Schéma du traitement progressif proposé



Mise à jour après échange : l'utilisateur a choisi Gemma 4 E2B mobile, priorité à la qualité, pour le prochain fine-tuning. Le nouveau corpus reprend V3 et ajoute les capacités ci-dessous. Les comparaisons de modèles conservées dans ce dossier documentent la recherche préalable ; ce choix fixe maintenant la priorité du pilote.

Décision proposée

Conserver Nemotron pour la reconnaissance vocale et entraîner un seul modèle de post-traitement avec trois modes explicites : `corrected`, `list`, `email`. Ce modèle reçoit de petits passages avec leur contexte, les corrige et les structure pendant que la dictée continue. L'application conserve la responsabilité des révisions, des corrections manuelles et de la publication du texte.

Gemma 4 E2B est le candidat retenu pour le prochain pilote, conformément au choix de l'utilisateur. Gemma 3 270M IT reste un témoin de comparaison, pas un choix de qualité déjà validé. Google le destine notamment aux tâches spécialisées de structuration de texte [S1]. Les autres familles examinées ci-dessous restent des alternatives si le pilote manque de fidélité ou dépasse le budget de latence. Le nouvel adaptateur part de Gemma 4 E2B : les données V3 sont réutilisables, les poids LoRA du 270M ne sont pas transférables entre ces architectures. Trois modèles chargés séparément ne sont pas nécessaires.

Le mail reste fidèle à la dictée : aucune salutation, signature, identité ou information ajoutée. L'objet n'apparaît que s'il est dicté comme tel. Le mode est choisi dans l'application ; prononcer « ignore les instructions » reste du contenu à transcrire.

Le mode Texte corrigé autorise aussi des listes locales : conserver les paragraphes avant et après une énumération claire, présenter ses éléments en puces, ou les numéroter si un ordre est explicitement dicté. Il ne transforme pas toute la dictée en liste. Quantités, conditions et compléments restent attachés au bon élément. Une coordination ordinaire, une citation ou une suite encore ambiguë reste en prose. En phase partielle, aucun item incomplet ne doit être complété en anticipant la suite.

Retour d'essai : priorité aux reprises orales

L'utilisateur rapporte un nettoyage insuffisant des répétitions, des changements de déterminant et des conjonctions abandonnées, avec le dernier APK GitHub Actions. Le dernier build réussi consulté est le run `35458263066`, du 19 septembre à 17 h 29 UTC, branche `codex/gemma270-v3-pilot`, révision `e8249e5`, artefact `dictai-gemma270-v3-test` [S10]. Son manifeste embarque bien Gemma 3 270M V3 LoRA, export Q8_0 de 291 545 280 octets, SHA-256 `6c4b7b6654c9638287c31e50fd0bf849f33a2ff2dd93bee685bdd9632f20ecf5`. L'installation exacte sur le téléphone n'est pas inspectée, mais cette provenance correspond au récit. Ce retour impose de réussir les réparations avant d'étendre un candidat aux trois formats.

Le corpus V3 comporte 436 exemples d'entraînement, dont huit étiquetés `local_gender_correction`. Les exemples inspectés de cette catégorie emploient des réparations explicites comme « un ... enfin une ... ». La recherche ciblée des formes adjacentes `le la`, `la le`, `un une`, `une un`, `du de la`, `pour le pour les` et des reprises `et mais`, `mais et`, `ou non`, `donc enfin` n'a trouvé aucune occurrence. Ce relevé limité ne prouve pas l'absence de toute disfluence, mais montre un défaut de couverture des constructions rapportées.

Deux causes principales restent à départager ici : couverture des données et capacité du modèle. Le code de cet APK utilise le prompt V3, un décodage greedy sur CPU à deux threads et accepte uniquement le français en mode corrigé. Son chemin pilote contourne `GemmaConservativeEditing` et vérifie surtout le transport, les termes protégés et les marqueurs d'image. Le filtre conservateur de la branche normale ne peut donc pas expliquer à lui seul cet essai. Conserver néanmoins les sorties brutes et les résultats affichés pour mesurer les replis, les délais et les différences après export. Ne pas conclure à une incapacité générale du 270M à partir d'un essai, ni supposer qu'un plus gros modèle suffira.

Exemples synthétiques de diagnostic : « je prends le la chemise bleue » doit devenir « Je prends la chemise bleue. » ; « c'est pour le pour les parents » doit devenir « C'est pour les parents. ». À l'inverse, conserver « très très content », « mais donc, pourquoi attendre ? », les citations et une fin provisoire telle que « je voulais le ». Supprimer tous les mots répétés ou toutes les suites de conjonctions serait une régression.

Le comparatif de développement prévoit : 270M V3 avec son prompt d'entraînement, même checkpoint avec consigne dédiée aux reprises, Qwen3-0.6B sans mode de réflexion [S8], puis Gemma 3 1B IT [S9] si l'accès existant l'autorise. Il n'entraîne aucun modèle. Les sorties sont jugées sur les réparations, les contre-exemples à préserver et les préfixes inachevés. L'exact match lexical est un diagnostic, pas une mesure complète de fidélité. Les chronométrages Mac ne permettent pas de prédire ceux du Poco ou de la tablette.

Une autre piste mérite une comparaison si la génération reste trop lente : un petit modèle spécialisé qui étiquette les mots à conserver, à retirer ou à laisser en attente. Le travail « Teaching BERT to Wait » étudie précisément le compromis entre contexte supplémentaire, qualité et délai en détection progressive des disfluences [S7]. Cela étaye la technique, sans valider son efficacité en français ni sur nos appareils. Un tel détecteur peut traiter les reprises ; il ne remplace pas à lui seul la correction lexicale et la mise en forme d'e-mails. Il reste une alternative de prototype, pas un composant ajouté à l'application.

Choisir le plus petit candidat qui passe les critères de fidélité et de latence mesurés. Si aucun ne les passe, enrichir le corpus ou essayer la voie spécialisée ; ne pas faire du 270M une contrainte de produit ni passer directement à un modèle beaucoup plus lourd sans comparaison.

Modèles locaux à comparer après ce diagnostic

Candidat	Intérêt pour DictAI	Limite à vérifier
Gemma 4 E2B, quantification mobile LiteRT-LM	Candidat Google prioritaire : runtime Android déjà présent dans la branche normale ; 2,3 milliards de paramètres effectifs, 5,1 milliards avec embeddings [S11].	Fidélité sur nos reprises et concurrence avec Nemotron, pas les seuls benchmarks généraux.
Qwen3.5-0.8B	Candidat plus petit et récent ; mode sans réflexion par défaut [S13].	Compatibilité du backend avec son architecture hybride ; qualité des suppressions et fin de génération.
Qwen3.5-2B	Palier à comparer si le 0.8B manque de capacité [S13].	Gain réel de qualité face au coût mémoire et à la latence supplémentaire.
Llama 3.2 1B Instruct	Témoin compact, français officiellement pris en charge [S14].	Aucune preuve actuelle de supériorité pour ces dictées ; le 3B est un palier plus coûteux.

Gemma 4 E2B existe bien dans une variante optimisée pour mobile. Attention à la mesure annoncée de 0,8 à 0,84 Go : elle concerne l'empreinte des poids en usage texte, pas un téléchargement de cette taille ni la mémoire totale de l'application. La fiche LiteRT précise l'usage du memory mapping pour 1,12 Go d'embeddings et le chargement des encodeurs audio/vision à la demande. Le fichier Android documenté reste d'environ 2,58 Go [S12]. La branche normale de DictAI cible déjà cet artefact : il faut comparer son comportement et son contrat de nettoyage, pas présenter une nouvelle variante « 840 Mo » comme remplacement acquis.

Google documente aussi la conversion de poids Gemma 4 personnalisés après fine-tuning vers LiteRT-LM et l'accélération MTP [S12]. La fidélité doit être revérifiée après conversion et quantification ; une compression arbitraire ne reproduit pas automatiquement les propriétés du modèle QAT officiel. Les mesures constructeur concernent notamment un S26 Ultra, pas le Poco F7 ou la Pad 7, et ne mesurent pas notre concurrence avec l'ASR. Les candidats ci-dessus ne sont pas encore comparés dans ce dossier ; la sonde de développement ci-avant porte sur les anciens modèles 270M/0.6B/1B disponibles dans le pipeline Python.

Ce que fait réellement le code actuel

Constats vérifiés dans le checkout `26cb80d`, avec seulement des documents non suivis au début de cette recherche :

- `LiveStreamingTranscriber.kt` transmet déjà un couple `committed/tentative`. Le backend GGUF le fournit ; le backend sherpa transmet actuellement tout comme provisoire.
- `OverlayService.scheduleLocalFormatting()` prépare une requête après 1 seconde sans changement. Chaque nouveau texte reporte ce départ. Une dictée continue peut donc empêcher tout départ avant l'arrêt.
- `LocalFormattingSession` limite les travaux à un actif et un en attente, mais sa préparation anticipée reste invisible. Le résultat n'est réutilisable que si le texte complet et le format correspondent exactement.
- Toute correction manuelle coupe cette préparation. `EditableTranscript` protège alors le préfixe affiché ; ce mécanisme ne représente pas encore des zones corrigées et des zones générées indépendantes.
- À l'arrêt, le texte final est retraité. `GEMMA_EDITING` n'affiche pas les tokens générés avant validation. Cette prudence doit rester vraie à l'échelle de chaque passage.
- Dans cette branche normale, `GemmaConservativeEditing` n'autorise qu'un ensemble borné de corrections de surface. Une correction contextuelle apprise peut être rejetée par ce filtre ; améliorer le modèle sans adapter son contrat de validation ne suffira pas. `GemmaFaithfulLayout` peut aussi restaurer certains mots omis : mesurer la sortie brute et celle retenue séparément. Le pilote APK décrit plus haut contourne ce filtre.
- `LocalFormatEngine` et `GemmaModelStore` ciblent Gemma 4 E2B LiteRT-LM, fichier d'environ 2,59 Go. L'entraînement Gemma 3 270M réside dans `/Users/ulliemaillot/dev_project/asr-postclean-fr` ; son intégration Android est livrée sur la branche pilote distincte `e8249e5`.

Les rapports locaux V3 donnent 61/76 correspondances lexicales, mais seulement 1/12 dans la catégorie de correction contextuelle. La revue sémantique consigne 58 réussites, 15 échecs et 3 cas ambigus, dont 6 déformations critiques introduites. V3b atteint 0/12 dans cette catégorie. Ces observations de validation ne démontrent pas une acceptation à 80 %. L'export GGUF V3 existe et passe quatre comparaisons simples après normalisation du suffixe ; cela ne valide ni les trois formats ni la performance sur téléphone.

Sources locales : `reports/v3/validation_review.json`, `reports/v3/semantic_validation.json`, `reports/v3b/validation_review.json` et `exports/gemma3-270m-v3-mobile/validation_report.json` dans le projet de fine-tuning. Aucun appareil n'est présent dans `adb devices` lors de la recherche.

Fonctionnement attendu

Voix → Nemotron → texte provisoire → passages suffisamment stables → modèle multiformat → validation → overlay modifiable. La capture et la reconnaissance continuent pendant le travail du modèle. L'injection dans l'application destinataire conserve son déclenchement actuel ; un aperçu ne doit jamais être injecté prématurément.

1. Afficher immédiatement les nouveaux mots comme provisoires. Distinguer discrètement la fin encore en reconnaissance et les passages traités ; ne pas faire reposer la distinction sur la couleur seule.
2. Choisir un passage à une frontière de phrase, d'item ou de paragraphe. Une frontière « stable » est une hypothèse révisable, jamais une preuve de correction. Les nombres, adresses, citations et quantités avec leurs compléments restent ensemble.
3. Transmettre le passage modifiable, un contexte précédent en lecture seule, le mode, la phase `partial/final` et les termes protégés. Le modèle retourne uniquement le passage traité. Il ne réécrit ni le contexte ni tout le document.
4. Publier le passage complet après validation, en une modification ciblée. L'utilisateur peut le corriger pendant que l'ASR poursuit la transcription et que les passages suivants sont traités.
5. Enregistrer toute frappe, suppression, collage et opération d'annulation comme une modification prioritaire. Une sortie calculée sur une ancienne version devient caduque. Le traitement des autres passages continue.
6. À l'arrêt, attendre le texte ASR final, réconcilier les différences, puis traiter la fin et les seuls passages invalidés. Une petite dictée peut n'avoir aucun traitement anticipé et suivre directement cette étape.

Point décisif de l'API native épinglée : `committed_text` est un préfixe d'affichage append-only, mais `full_text` reste la transcription brute de référence, y compris après finalisation [S3]. Ne jamais considérer `committed + tentative` comme nécessairement identique au texte final. Le callback Kotlin devra transporter aussi la transcription complète et une version monotone ; les offsets natifs en octets UTF-8 ne doivent pas être confondus avec les positions UTF-16 de l'éditeur Android.

Contrat de concurrence et d'édition

Chaque requête transporte au minimum `runId`, `formatVersion`, `segmentId`, `sourceRevision`, `contextRevision` et `editRevision`. Une réponse ne peut être appliquée que si ces versions correspondent encore. Une nouvelle dictée, un changement de format, une annulation, une correction ou une révision du contexte invalide les réponses concernées.

Le document conserve séparément la transcription brute, les propositions du modèle et les modifications de l'utilisateur. Les verrous portent sur des intervalles ancrés au texte source, avec une trace des suppressions ; une suppression de longueur finale nulle doit rester protégée. Une correction d'un mot ne bloque pas toute la suite de la dictée. Le contexte fourni au modèle contient la version corrigée par l'utilisateur, en lecture seule.

Pendant une composition clavier IME, différer toute publication qui touche la zone composée. Préserver sélection, curseur, position de lecture, images et annulation. Une autocorrection orale tardive peut réviser une zone non modifiée manuellement. Si elle entre en conflit avec un verrou humain ou désigne un passage de façon ambiguë, conserver la modification humaine et présenter la correction comme suggestion, sans remplacement silencieux.

Un seul travail LLM actif et une seule demande en attente, remplaçable. Ne pas relancer l'intégralité du texte à chaque mot. Ne pas annuler un passage encore utile parce qu'un autre s'allonge. Ne pas accumuler de retard non borné. La capture audio et le décodage ASR sont prioritaires ; aucun travail natif sur le thread UI/audio.

Paramètres de départ à mesurer : passages d'environ 24 à 96 mots, stabilité observée pendant 0,8 à 1,5 seconde, contexte précédent borné. Un âge maximal de demande doit empêcher le debounce de repousser indéfiniment le travail, sans forcer la validation d'une phrase incomplète. Pour sherpa sans préfixe stable, utiliser l'accord entre plusieurs hypothèses et conserver davantage de texte provisoire. Ces valeurs ne sont pas des performances mesurées.

Pour conserver le contexte global sans régénérer toute la dictée, sélectionner aussi des extraits exacts antérieurs pertinents, avec leurs identifiants et versions : nom déjà correctement écrit, thème explicite, élément de liste ou destinataire. Ne pas remplacer ces preuves par un résumé inventé. Si le contexte utile ne tient pas dans la fenêtre, conserver le passage ambigu et le réexaminer à la finalisation.

Un manque de mémoire, un retard ASR ou une surchauffe suspend le travail anticipé ; le mode final reste disponible. Conserver le texte source et les corrections humaines si le modèle échoue. Ne pas basculer vers le cloud automatiquement.

Les termes du vocabulaire personnalisé restent corrigés de façon déterministe avant le modèle, puis protégés par le code. La présence d'un terme quelque part dans la sortie ne suffit pas à vérifier ses occurrences ni son rôle. Protéger les correspondances et leur position logique. Les contrôles automatiques de nombres et de négations ne constituent pas un juge sémantique. Une réparation explicite telle que « 15, non 16 » autorise de retenir 16 ; une disparition du 15 sans cette réparation est une déformation. Ces exceptions doivent être annotées dans les cibles et testées, sans assouplir globalement les protections.

Données du fine-tuning

Le corpus V4 reprend les 436 exemples d'entraînement et 76 de validation de V3, qui incorporent déjà les versions précédentes : aucune seconde importation de V1/V2. Les partitions et les textes d'origine sont conservés avec leur provenance. Les nouveaux épisodes ajoutent les trois modes, les deux phases et la mise en forme mixte. Les anciens tests réservés demeurent hors entraînement ; une vérification automatisée des collisions ne doit pas exposer leurs cibles ni servir à créer les nouveaux exemples.

L'assemblage réalisé contient 604 exemples d'entraînement, 124 de validation et 24 de test de développement. Les ajouts représentent 40 épisodes synthétiques, déclinés en 240 exemples et relus avant le gel. Les données V3 sont préservées à l'identique. L'audit automatique n'a trouvé aucune collision normalisée entre les ajouts et les deux tests réservés V3. Ces contrôles ne remplacent pas une évaluation sémantique ni une validation sur des dictées réelles ; les 24 nouveaux cas ne sont pas un benchmark aveugle indépendant.

Entraîner conjointement les trois modes. Conserver suffisamment de nettoyage simple et de textes déjà corrects pour détecter la régression des capacités existantes. Pour le pilote Gemma 4 sur le Mac de 16 Go, la voie retenue est MLX-LM 0.31.3 avec des poids communautaires quantifiés en 4 bits [S15]. Réglages initiaux : huit dernières couches, projections d'attention Q et O, rang 16, multiplicateur MLX `scale=2.0` (équivalent au rapport PEFT `alpha/r=32/16`), taux d'apprentissage `5e-5`, batch 1, accumulation sur quatre microbatches, checkpointing des gradients et graine 42. Vérifier les longueurs réelles avant entraînement ; la limite de 2 048 tokens ne doit jamais tronquer silencieusement les cibles. La pondération des éditions ×10 de V3b n'a pas résolu le problème contextuel : ne pas la reprendre par défaut.

Le chargement et une génération courte de Gemma 4 E2B ont réussi localement dans cet environnement : environ 2,48 Gio de mémoire GPU MLX au pic pour ce seul essai. Cette mesure n'est ni la mémoire totale du Mac, ni celle de l'entraînement, ni une mesure mobile. La révision communautaire utilisée est `238767527555cb75a05732a84dff5d6ba0dd6809`, issue selon sa fiche de la révision Google `70af34e20bd4b7a91f0de6b22675850c43922a03`.

Le corpus doit contenir des épisodes chronologiques : hypothèses ASR successives, pauses, début de phrase inachevé, ajout de la suite, autocorrection et édition humaine. Pour chaque état partiel, la cible utilise uniquement le texte déjà disponible. Une ponctuation ou un complément connu seulement après la suite ne doit pas apparaître dans une cible antérieure.

Champs du premier contrat : `id`, `scenario_id`, `episode_id`, `mode`, `phase`, `source`, `context_before`, `protected_terms`, `target`, `tags`, `synthetic`, `reviewed`. `source` est le segment modifiable ; `target` est uniquement son rendu. La simulation des événements et des verrous Android sera un corpus de replay distinct, relié aux mêmes scénarios.

Le kit initial utilise un contexte précédent. Pour réexaminer un ancien passage après une correction orale tardive, le futur contrat devra aussi transporter les extraits déjà connus au moment de cette révision, avec leur ordre temporel et leurs versions ; ils ne constituent pas de l'information future pour cette nouvelle requête.

Séparer train/validation/test par scénario et épisode, avant expansion en formats, variantes et préfixes. Les familles de gabarits et les quasi-doublons doivent aussi être audités avant la campagne réelle. Aucun recours aux holdouts V1-V3 pour produire de nouveaux exemples d'entraînement. Ne pas utiliser les dictées privées comme corpus automatique.

Le kit synthétique initial sert à tester ce contrat et les exports. Il ne constitue pas un corpus suffisant. Budget initial proposé : 1 500 à 3 000 épisodes variés, à ajuster à partir des erreurs observées ; conserver au moins 100 épisodes de test indépendants par format. Ce sont des budgets de collecte, pas une garantie statistique ni une obligation de fabriquer des milliers de paraphrases presque identiques.

Un export conversationnel `prompt/completion` permet de calculer la loss sur la réponse seule [S4]. Utiliser le chat template exact du tokenizer, masquer contexte et consignes, vérifier les longueurs sans troncature silencieuse. Éviter de demander à un 270M de produire à la fois de la prose et une structure complexe de patches : les patches et leur validation sont construits par l'application.

Matrice des cas à couvrir

Famille	Cas indispensables
Fidélité	Noms propres, vocabulaire personnalisé, nombres, montants, dates, heures, négations, conditions, incertitudes, contenu cité, mots répétés volontairement.
Réparations	Hésitations, répétitions accidentelles, « mardi, non jeudi », correction plusieurs phrases plus tard, ambiguïté à conserver, erreur phonétique résolue seulement avec le contexte.
Listes	Un item, items sans virgules, quantités et compléments, introduction, actions avec échéances, sous-groupes explicites, changement d'item encore incomplet, passage déjà en liste.
E-mails	Corps seul, salutations dictées, objet dicté, destinataire et signataire différents, signature absente, réponse très courte, plusieurs sujets, liste dans le corps, citation et adresse e-mail.
Progressif	Débit continu, pauses, absence de ponctuation, hypothèse révisée, suffixe inachevé, contexte tardif, final ASR différent de l'aperçu, phrase qui dépasse la fenêtre.
Interaction	Frappe pendant génération, effacement, collage, sélection, IME, undo/redo, changement de format, pause/reprise, annulation, nouvelle dictée, récupération de brouillon, marqueurs d'image.
Exécution	Silence, texte vide, timeout, sortie tronquée, tokens parasites, fin de génération tardive, RAM, chauffe, ASR batch sans aperçu, français/anglais et alternance de langue.

Cette matrice organise la couverture ; elle ne promet pas tous les cas possibles. Les trois formats doivent être évalués séparément, avec des dictées courtes et longues et des thèmes absents de l'entraînement.

Évaluation et passage sur téléphone

Comparer sur les mêmes scénarios : pipeline actuel, candidat multiformat final uniquement, puis candidat multiformat progressif. Mesurer à la fois les sorties brutes du modèle et le résultat réellement accepté par l'application. Un bon résultat de repli ne doit pas masquer un taux de rejet élevé du modèle.

Critères proposés : au moins 80 % d'acceptabilité sémantique humaine dans chaque format et pour partial/final ; au moins 95 % de structure correcte sur les cas où la structure est attendue ; aucune corruption des verrous humains, des nombres, des négations et du vocabulaire protégé dans la suite de livraison. Rapporter numérateurs, dénominateurs et incertitude ; un zéro observé ne garantit pas un risque nul. Ne pas confondre exact match lexical, loss et fidélité sémantique.

Mesures Poco F7 et Pad 7 : RTF ASR seul et concurrent, délai premier passage validé, attente après arrêt p50/p95, taux de réutilisation, reprises de segments, taux de rejet/repli, mémoire totale, énergie et comportement après plusieurs minutes. Cibles de départ à valider : réduire d'au moins 30 % l'attente p95 sur dictées longues sans dégrader le RTF ASR de plus de 10 %. Si elles ne sont pas atteintes, diminuer la concurrence ou garder le traitement final.

Fusionner l'adaptateur LoRA dans le modèle avant export est possible avec PEFT [S5]. Comparer le checkpoint original, le modèle fusionné, le modèle quantifié et le runtime Android avec exactement le même template et les mêmes arrêts. L'export GGUF déjà disponible est une voie de test ; il ne peut pas être chargé tel quel par l'actuel moteur LiteRT-LM. Google documente aussi la conversion d'un Gemma 270M personnalisé vers LiteRT-LM et son exécution Android [S6]. Choisir le backend après comparaison CPU/GPU sur les appareils, sans promettre d'accélération sur la seule base des formats supportés.

Pour le nouveau pilote Gemma 4, l'adaptateur MLX n'est pas directement un fichier mobile. L'export GGUF de MLX-LM 0.31.3 ne prend pas en charge Gemma 4 [S16]. La conversion Google de poids Hugging Face personnalisés vers LiteRT-LM [S12] exige donc une étape distincte de fusion et de correspondance des poids, puis des tests de parité. Les poids communautaires 4 bits du Mac ne sont pas l'artefact QAT mobile officiel ; leurs qualités et empreintes ne doivent pas être confondues.

Sources primaires consultées

- S1 - Google, Gemma 3 270M et tâches spécialisées.
- S2 - NVIDIA, modèle Nemotron 3.5 ASR multilingue streaming. Le français est pris en charge ; les mesures H100 de cette fiche ne prédisent pas la latence Android.
- S3 - transcribe.cpp, contrat C à la révision embarquée 553f109, sections `transcribe_stream_text` et politique de commitment.
- S4 - Hugging Face TRL, SFT Trainer et completion-only loss.
- S5 - Hugging Face PEFT, fusion LoRA.
- S6 - Google AI Edge, déployer un Gemma 270M fine-tuné et conversion des modèles Hugging Face.
- S7 - Chen et al., Teaching BERT to Wait, NAACL 2022, détection progressive des disfluences avec attente adaptative.
- S8 - Qwen, Qwen3-0.6B et désactivation du mode de réflexion.
- S9 - Google, Gemma 3 1B IT.
- S10 - GitHub Actions, APK pilote Gemma 270M V3, manifeste et code inspectés à la révision `e8249e59f43cb4b6826fff471c669cd03db0a15b`.
- S11 - Google, caractéristiques Gemma 4 et quantifications mobiles.
- S12 - Google AI Edge, Gemma 4 sous LiteRT-LM et fiche du fichier Android.
- S13 - Qwen3.5-0.8B et Qwen3.5-2B, fiches officielles.
- S14 - Meta, Llama 3.2 1B Instruct.
- S15 - MLX-LM 0.31.3, fine-tuning LoRA et QLoRA et conversion communautaire Gemma 4 E2B 4 bits.
- S16 - MLX-LM 0.31.3, formats autorisés par l'export GGUF.

La séparation des zones, l'ordonnancement, les seuils, le budget de données et les critères de passage ci-dessus sont des décisions de conception proposées pour DictAI, pas des promesses des sources.